

Sistema de Orçamento Imobiliário R.M.

Pensamento algorítmico, orientação a objetos

Relatório técnico do projeto

Instituição: Faculdade UniFECAF

Disciplina: Algorithmic Thinking & Introduction to Object-Oriented Programming

Aluno: Antônio Benevaldo Chaves Santana

Professor: Afonso Cesar Lelis Brandão

Curso: Ciência de Dados

Sumário

1. Introdução	2
2. Modelagem do problema	2
3. Requisitos funcionais e não funcionais	4
4. Análises estática e dinâmica	6
5. Modelagem de dados	7
6. Arquitetura em camadas	8
7. Pensamento algorítmico	9
8. Fluxograma do cálculo principal	10
9. Regras de negócio e memória de cálculo	11
10. Programação orientada a objetos	12
11. TDD: RED, GREEN e REFACTOR	13
12. Implementação tecnológica	15
13. Integridade SQLite	16
14. Geração do CSV com 12 meses	17
15. Testes e validação	18
16. Execução do projeto	19
17. Entregáveis	19
18. Conclusão	20
19. Referências	20

1. Introdução e objetivos

O Sistema de Orçamento Imobiliário R.M foi desenvolvido para automatizar a geração de orçamentos mensais de locação de apartamentos, casas e estúdios. A solução recebe os dados necessários, aplica regras específicas para cada imóvel, calcula os encargos contratuais, registra o orçamento em banco de dados e permite exportar um cronograma de 12 meses em CSV.

O projeto foi estruturado como um monólito modular em Python com interface Streamlit. A camada de apresentação delega os cálculos a serviços que utilizam modelos orientados a objetos, enquanto a camada de persistência realiza o registro transacional no SQLite.

1.1 Objetivo geral

Desenvolver uma aplicação funcional, organizada e verificável que transforme as regras comerciais da Imobiliária R.M em uma sequência lógica de operações, demonstrando os conceitos de pensamento algorítmico e programação orientada a objetos estudados na disciplina.

1.2 Objetivos específicos

Calcular o aluguel mensal conforme o tipo de imóvel e as opções escolhidas.

Aplicar acréscimos e descontos somente quando as respectivas condições forem satisfeitas.

Calcular o contrato imobiliário de R\$ 2.000,00, parcelável em até cinco vezes.

Gerar um cronograma financeiro de 12 meses e disponibilizá-lo em CSV.

Organizar as regras por meio de classes, herança, abstração e composição.

Persistir os orçamentos com integridade referencial em banco SQLite.

Validar regras, banco de dados, interface e exportação mediante testes automatizados.

2. Modelagem do problema e requisitos

A modelagem do problema representa a etapa de compreensão do problema proposto no enunciado. Antes da implementação, as regras apresentadas no enunciado foram transformadas em objetivos, entradas, restrições, decisões e resultados esperados. Essa etapa reduz ambiguidades e permite validar o entendimento do problema antes de escrever o código.

2.1 Problema identificado

A elaboração manual dos orçamentos exige a consulta de diferentes preços e condições comerciais, aumentando o risco de aplicar acréscimos ou descontos incorretos. A empresa precisa de uma aplicação que padronize os cálculos e apresente ao cliente uma previsão financeira de 12 meses.

2.2 Necessidades identificadas no enunciado

Trabalhar com apartamentos, casas e estúdios, cada um com seu valor-base.

Aplicar acréscimos de quartos e vagas conforme o tipo de imóvel.

Conceder desconto de 5% somente no aluguel de apartamentos para clientes sem crianças.

Adicionar o contrato imobiliário de R\$ 2.000,00, parcelável em até cinco vezes.

Exibir com clareza a composição do aluguel, do contrato e do total do primeiro ano.

Gerar um arquivo CSV com o cronograma das 12 parcelas mensais.

Manter os orçamentos registrados para consulta e auditoria.

2.3 Entradas, processamento e saídas

Elemento	Descrição
Entradas	Cliente, presença de crianças, tipo de imóvel, quartos, vagas e parcelas do contrato.
Processamento	Validação, seleção das regras, cálculo monetário, cronograma, persistência e exportação.
Saídas	Aluguel mensal, memória do cálculo, contrato, total anual, registro SQLite e CSV de 12 meses.
Restrições	Parcelas entre 1 e 5; valores não negativos; combinações de quartos e vagas válidas.

2.4 Critérios de aceitação definidos

Os três cenários oficiais devem produzir R\$ 1.140,00, R\$ 1.450,00 e R\$ 1.570,00.

O contrato de R\$ 2.000,00 deve respeitar o limite de cinco parcelas.

O CSV deve possuir exatamente 12 registros mensais, além do cabeçalho.

Os valores exibidos, persistidos e exportados devem ser idênticos.

Uma falha de validação ou banco não pode deixar um orçamento parcialmente registrado.

3. Requisitos funcionais e não funcionais

Os requisitos funcionais descrevem o que a aplicação deve realizar. Os requisitos não funcionais estabelecem as características de qualidade, segurança, desempenho e manutenção esperadas.

3.1 Requisitos funcionais

Código	Requisito funcional
RF01	Cadastrar ou informar os dados necessários do cliente sem utilizar dados reais na demonstração.
RF02	Selecionar apartamento, casa ou estúdio e suas opções permitidas.

RF03	Calcular o aluguel conforme preços, acréscimos e desconto definidos.
RF04	Calcular o contrato de R\$ 2.000,00 em até cinco parcelas.
RF05	Exibir a memória do cálculo e o total do primeiro ano.
RF06	Persistir cliente, imóvel, contrato, orçamento, itens e parcelas em transação.
RF07	Gerar e disponibilizar o CSV com o cronograma de 12 meses.
RF08	Permitir consultar os orçamentos registrados.

3.2 Requisitos não funcionais

Código	Categoria	Requisito não funcional
RNF01	Precisão	Representar valores monetários em centavos inteiros, sem FLOAT.
RNF02	Usabilidade	Apresentar campos, mensagens e resultados de forma clara e consistente.
RNF03	Desempenho	Concluir o cálculo e a gravação sem atraso perceptível em uso local.
RNF04	Confiabilidade	Utilizar transações e restrições para impedir registros parciais ou inválidos.
RNF05	Manutenibilidade	Separar interface, modelos, serviços, persistência e exportação.
RNF06	Portabilidade	Executar em Python 3.10 ou superior com dependências documentadas.
RNF07	Testabilidade	Permitir testes isolados das regras, do banco e da interface.
RNF08	Privacidade	Evitar dados pessoais reais nos testes, capturas e demonstrações.

3.3 Especificação detalhada das regras funcionais

Item	Regra
Apartamento	R\$ 700,00 por mês para uma unidade com um quarto.
Casa	R\$ 900,00 por mês para uma unidade com um quarto.
Estúdio	R\$ 1.200,00 por mês.
Contrato	R\$ 2.000,00, parcelável em até cinco vezes.
Apartamento com 2 quartos	Acréscimo de R\$ 200,00.
Casa com 2 quartos	Acréscimo de R\$ 250,00.

Garagem de casa/apartamento	Acréscimo de R\$ 300,00.
Estúdio com estacionamento	R\$ 250,00 pelas duas primeiras vagas e R\$ 60,00 por vaga adicional.
Desconto	5% no aluguel do apartamento quando o cliente não possui crianças.
Saída	Exibir orçamento mensal, contrato e gerar CSV com 12 meses.

4. Modelagem estática e modelagem dinâmica

A modelagem estática descreve os elementos que formam o sistema e seus relacionamentos. A modelagem dinâmica demonstra como esses elementos colaboram durante a execução de um orçamento.

4.1 Modelagem estática

A visão estática é representada principalmente pelo diagrama de classes apresentado na Figura 2. As classes Cliente, Imovel, Apartamento, Casa, Estudio, Contrato, ItemCalculo e Orcamento representam o domínio. CalculoService e ExportService coordenam casos de uso sem transferir as regras para a interface.

Elemento estático	Responsabilidade
Cliente	Fornecer os dados necessários para aplicar as condições comerciais.
Imovel	Definir a abstração comum e o contrato de cálculo.
Apartamento, Casa e Estudio	Especializar as regras por herança e polimorfismo.
Orcamento	Agregar cliente, imóvel, contrato, itens e cronograma.
Serviços	Orquestrar o cálculo e a exportação sem depender da interface.
Banco SQLite	Persistir o estado completo do orçamento e seus relacionamentos.

4.2 Modelagem dinâmica

A visão dinâmica é representada pelo fluxograma da Figura 1 e pela sequência entre as camadas. O usuário informa os dados; a interface valida; o serviço seleciona a classe concreta do imóvel; o objeto calcula os itens; o orçamento é composto; o banco salva a transação; e o serviço de exportação gera o CSV. Decisões condicionais determinam qual regra será executada.

Receber e validar os dados do
usuário. Identificar o tipo de imóvel

selecionado.

Executar o comportamento polimórfico

correspondente. Calcular contrato e cronograma

de 12 meses.

Persistir o agregado completo em uma transação.

Apresentar o resultado e liberar a exportação.

4.3 Relação entre as duas modelagens

Síntese: A modelagem estática responde quais objetos existem e como se relacionam; a modelagem dinâmica responde em que ordem eles participam e como o estado do orçamento evolui durante a execução.

5. Modelagem de dados

A modelagem de dados transforma as entidades do domínio em uma estrutura relacional persistida no SQLite. O modelo foi organizado para evitar duplicidades, preservar a memória do cálculo e garantir que cada item e parcela permaneça associado ao orçamento correto.

5.1 Modelo conceitual

Um cliente pode possuir vários orçamentos.

Um imóvel pode aparecer em diferentes orçamentos ao longo do tempo.

Cada orçamento possui exatamente um contrato.

Cada orçamento possui um ou mais itens de cálculo.

Cada orçamento possui exatamente 12 parcelas mensais no cronograma.

5.2 Modelo lógico

Tabela	Finalidade	Relacionamento principal
clientes	Dados mínimos do solicitante.	1:N com orcamentos
imoveis	Tipo e características utilizadas no cálculo.	1:N com orcamentos
contratos	Valor, quantidade e valor das parcelas.	1:1 com orcamentos
orcamentos	Agregado principal e totais calculados.	N:1 com cliente e imóvel
itens_orcamento	Memória detalhada dos acréscimos e	N:1 com orcamentos

	descontos.	
parcelas_orcament o	Cronograma financeiro de 12 meses.	N:1 com orcamentos

5.3 Modelo físico

No modelo físico, valores monetários são armazenados como INTEGER em centavos. Chaves primárias identificam os registros, chaves estrangeiras asseguram os vínculos e restrições CHECK e UNIQUE impedem estados inválidos. A Figura 3 apresenta as tabelas, os campos e as cardinalidades implementadas.

5.4 Normalização e integridade

Cada tabela representa um assunto principal e evita grupos repetitivos.

Itens e parcelas são separados do orçamento para permitir múltiplos registros relacionados.

O contrato mantém relação exclusiva de um para um com o orçamento.

A exclusão em cascata é limitada aos itens e parcelas dependentes.

Entidades principais utilizam restrição de exclusão para evitar perda acidental.

6. Arquitetura

A solução utiliza uma arquitetura monolítica modular em camadas. Todos os módulos são executados como uma única aplicação, mas as responsabilidades permanecem separadas, facilitando testes, manutenção e evolução.

Camada	Componentes	Responsabilidade
Apresentação	app.py e pages/	Coletar entradas, exibir mensagens e apresentar resultados.
Aplicação	CalculoService e ExportService	Orquestrar casos de uso e coordenar o domínio.
Domínio	models/	Representar entidades e executar regras de negócio orientadas a objetos.
Persistência	database/ e SQLite	Controlar conexões, transações, consultas e integridade.

Camada	Componentes	Responsabilidade
Exportação	pandas e CSV	Transformar o cronograma em arquivo interoperável.
Testes	tests/	Validar regras, persistência, interface e exportação.

6.1 Decisões arquiteturais

Streamlit foi escolhido para fornecer uma interface web simples desenvolvida integralmente em Python.

As regras foram retiradas da interface e concentradas nos modelos e serviços.

SQLite foi adotado como banco transacional local, adequado ao escopo acadêmico.

pandas foi utilizado somente na preparação da exportação CSV.

Dependências entre camadas seguem a direção interface, aplicação, domínio e persistência.

6.2 Fluxo arquitetural

A interface recebe os dados e realiza validações básicas.

O serviço de cálculo seleciona e aciona o modelo

do imóvel. O domínio produz itens e totais sem

dependar do Streamlit.

A camada de persistência salva o agregado dentro de uma

transação. O serviço de exportação lê o cronograma

consolidado e gera o CSV.

7. Pensamento algorítmico

O pensamento algorítmico foi aplicado pela decomposição do problema em etapas menores, ordenadas e verificáveis. Cada etapa possui entradas, processamento e saídas definidos, reduzindo a complexidade e facilitando os testes.

7.1 Entradas

Dados essenciais do cliente, incluindo a informação sobre possuir ou não crianças.

Tipo de imóvel: apartamento, casa ou estúdio.

Quantidade de quartos, quando aplicável.

Quantidade ou existência de vagas de garagem/estacionamento.

Quantidade de parcelas do contrato, limitada ao intervalo de uma a cinco.

7.2 Processamento

Validar as entradas e impedir combinações

incompatíveis. Selecionar a regra correspondente

ao tipo de imóvel.

Inicializar o cálculo com o valor-base do imóvel.

Somar os acréscimos de quarto e estacionamento quando aplicáveis.

Aplicar o desconto de 5% somente ao apartamento cujo cliente não possui crianças. Calcular o contrato e o valor de cada parcela.

Montar o cronograma de 12 meses, incluindo as parcelas contratuais apenas nos primeiros meses contratados.

Salvar o agregado completo em uma transação SQLite e preparar o CSV.

7.3 Estruturas utilizadas

Estrutura	Aplicação no projeto
Sequência	Coleta, validação, cálculo, persistência, exibição e exportação.
Seleção	Decisões por tipo de imóvel, quantidade de quartos, vagas e presença de crianças.
Repetição	Geração dos 12 registros mensais do cronograma.
Decomposição	Separação entre interface, modelos, serviços, banco e exportação.
Abstração	Classe Imóvel define o comportamento comum dos diferentes imóveis.

7.4 Saídas

Valor mensal do aluguel e memória dos itens que formaram o cálculo.

Valor total do contrato, número de parcelas e valor de cada parcela.

Total do primeiro ano e cronograma mensal.

Registro persistido no SQLite.

Arquivo CSV com 12 linhas mensais, além do cabeçalho.

8. Fluxograma da aplicação

O fluxograma apresenta a sequência principal do programa e explicita as decisões condicionais exigidas para selecionar as regras de cada imóvel.

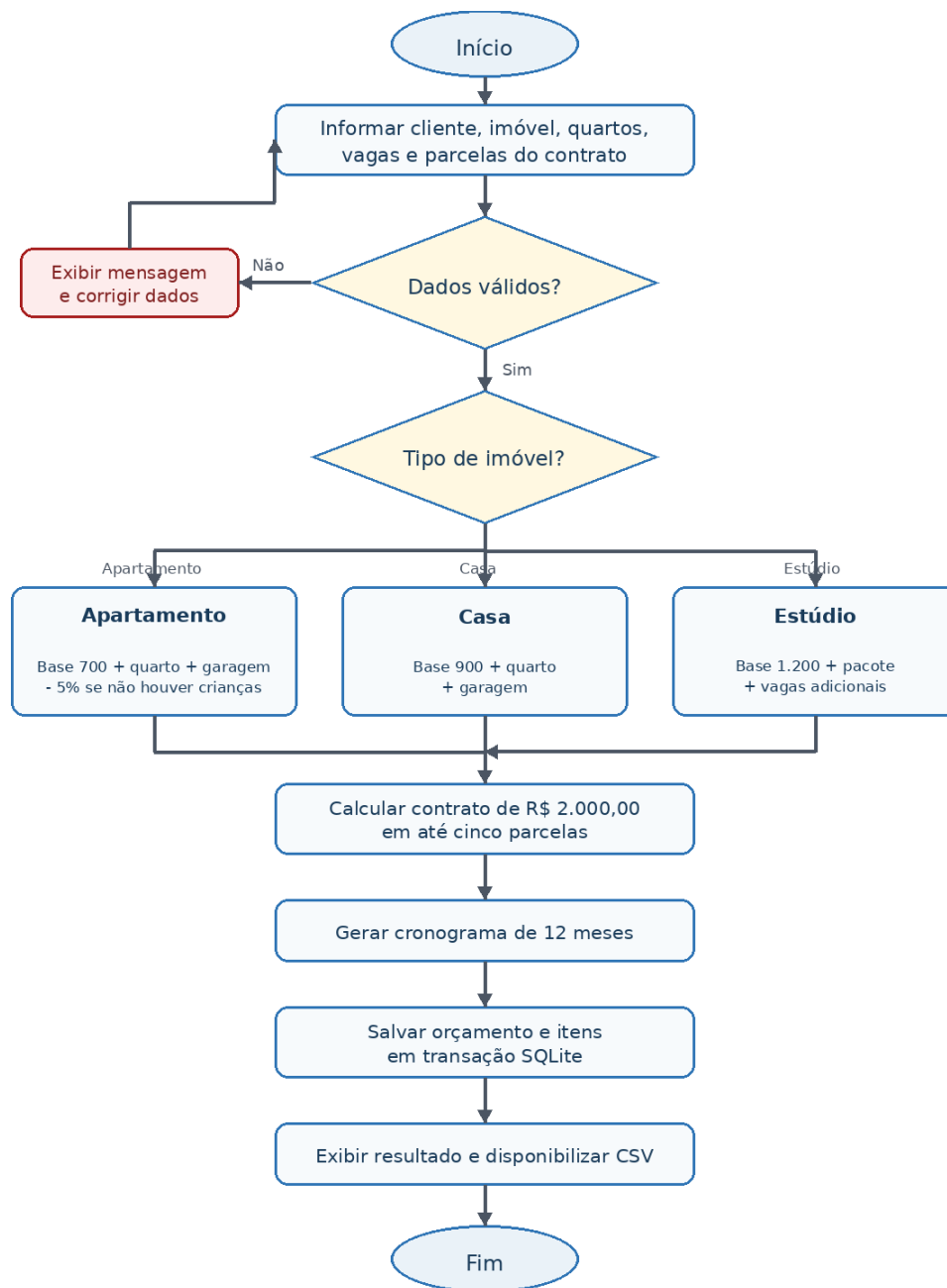


Figura 1 - Fluxograma geral do processo de orçamento

9. Regras de negócio e cálculos

9.1 Apartamento

Fórmula geral:

$\text{aluguel} = (700 + \text{adicional_quarto} + \text{adicional_garagem}) - \text{desconto}$

$\text{aluguel} = (700 + \text{adicional_quarto} + \text{adicional_garagem}) - \text{desconto}$

Segundo quarto: adicionar R\$ 200,00.

Garagem: adicionar R\$ 300,00.

Sem crianças: aplicar 5% sobre o subtotal do aluguel.

Exemplo oficial: R\$ 700,00 + R\$ 200,00 + R\$ 300,00 = R\$ 1.200,00. O desconto de 5% corresponde a R\$ 60,00, resultando em R\$ 1.140,00.

9.2 Casa

$$\text{aluguel} = 900 + \text{adicional_quarto} + \text{adicional_garagem}$$

Exemplo oficial: R\$ 900,00 + R\$ 250,00 + R\$ 300,00 = R\$ 1.450,00. O desconto de 5% não se aplica às casas.

9.3 Estúdio

$$\text{aluguel} = 1.200 + \text{pacote_duas_vagas} + \text{vagas_adicionais}$$

Para duas vagas, acrescentam-se R\$ 250,00. A partir da terceira, cada vaga adicional custa R\$ 60,00. No cenário com quatro vagas: R\$ 1.200,00 + R\$ 250,00 + 2 × R\$ 60,00 = R\$ 1.570,00.

Validação importante: A interface deve aceitar zero vaga ou pelo menos duas vagas para o estúdio, pois a regra fornecida não define preço isolado para somente uma vaga.

9.4 Contrato imobiliário

O contrato possui valor fixo de R\$ 2.000,00. A quantidade de parcelas deve estar entre uma e cinco. Quando dividido em cinco parcelas, o valor de cada parcela é R\$ 400,00.

9.5 Tratamento dos valores monetários

Todos os valores são representados internamente em centavos inteiros. Por exemplo, R\$ 700,00 é armazenado como 70000. Essa decisão evita imprecisões de ponto flutuante e mantém o código coerente com o modelo físico SQLite. A formatação em reais ocorre apenas na apresentação.

10. Orientação a objetos

A orientação a objetos organiza as responsabilidades e permite que cada tipo de imóvel mantenha suas próprias regras. A classe abstrata `Imovel` define o contrato comum, enquanto `Apartamento`, `Casa` e `Estudio` especializam o comportamento por herança e polimorfismo.

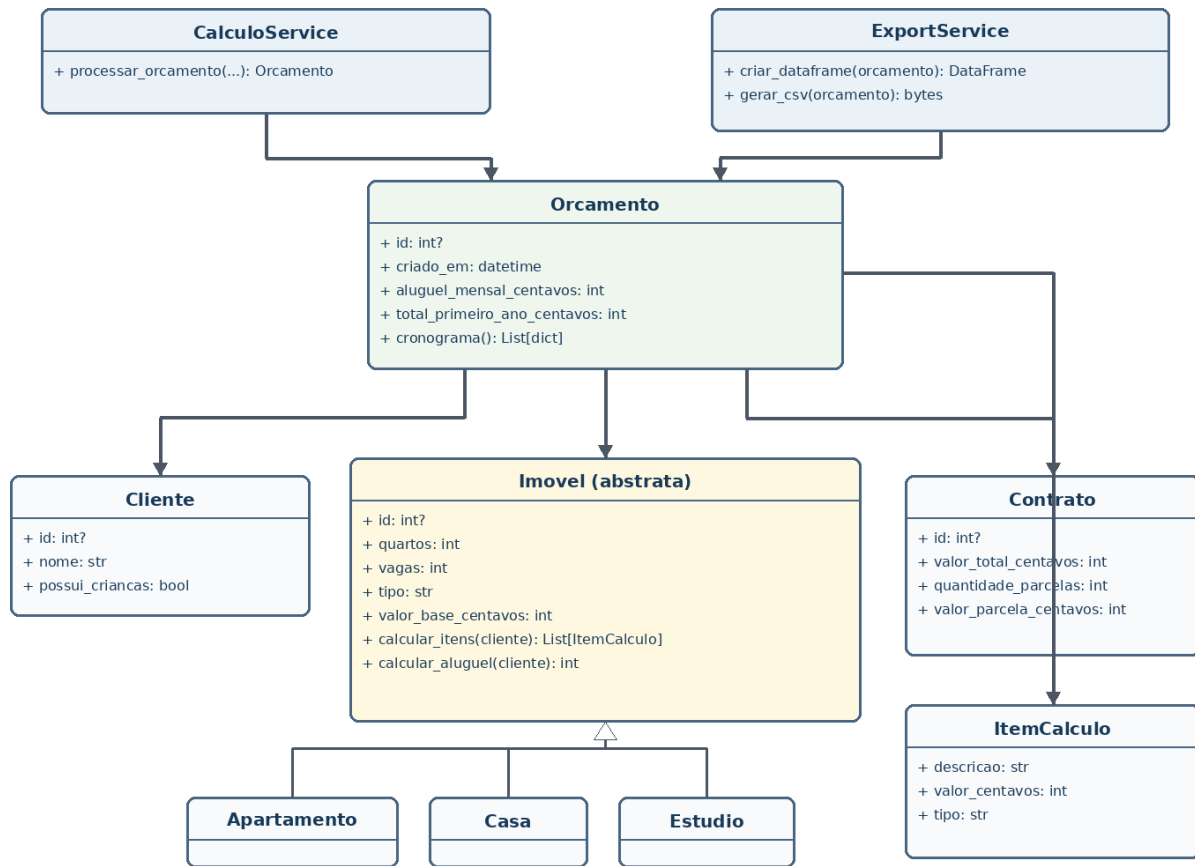


Figura 2 - Diagrama de classes corrigido com valores monetários em centavos inteiros

10.1 Princípios demonstrados

Princípio	Aplicação
Abstração	Imovel representa características e operações comuns.
Herança	Apartamento, Casa e Estudio derivam de Imovel.
Polimorfismo	Cada classe implementa seu próprio cálculo de aluguel e itens.
Encapsulamento	As regras permanecem dentro das classes e dos serviços responsáveis.
Composição	Orcamento reúne Cliente, Imovel, Contrato, ItemCalculo e cronograma.
Separação de responsabilidades	CalculoService calcula; ExportService exporta; OrcamentoRepository realiza a persistência.

11. TDD - Test-Driven Development

TDD é uma abordagem de desenvolvimento orientada por testes. Uma regra é descrita primeiro como um teste automatizado; em seguida, implementa-se apenas o código necessário para satisfazê-lo. No projeto, o ciclo pode ser demonstrado com as regras de apartamento, casa, estúdio, contrato e cronograma.

Importante: Para comprovar TDD no repositório, é recomendável preservar commits que mostrem o teste falhando antes da implementação e, depois, o mesmo teste aprovado.

11.1 Fase RED

Na fase RED, escreve-se um teste para um comportamento ainda não implementado ou ainda incorreto. O teste deve falhar pelo motivo esperado, demonstrando que a regra foi compreendida e que existe trabalho de implementação a realizar.

Escolher uma regra pequena e

objetiva. Definir entradas e resultado

esperado.

Criar o teste automatizado com uma asserção

clara. Executar pytest e confirmar a falha

esperada.

Registrar o resultado antes de alterar o código de produção.

Exemplo conceitual da fase RED:

```
def test_apartamento_completo_sem_crianças():
```

```
    assert calcular_aluguel(...) == 114000
```

```
# Resultado inicial esperado: teste falhando
```

11.2 Fase GREEN

Na fase GREEN, implementa-se a solução mínima capaz de fazer o teste passar. O objetivo não é aperfeiçoar toda a arquitetura imediatamente, mas atender corretamente à regra descrita pelo teste, sem quebrar os comportamentos já aprovados.

Implementar somente a regra necessária para o cenário

testado. Executar novamente o teste específico.

Executar toda a suíte para identificar regressões.

Comandos de verificação da fase GREEN:

```
python -m pytest tests/test_calculo.py -v
```

```
python -m pytest -v
```

```
# Resultado esperado: todos os testes aprovados
```

Confirmar que o resultado está correto em centavos

inteiros. Registrar a aprovação antes de iniciar uma nova

regra.

11.3 Fase REFACTOR e continuidade do ciclo

Após a fase GREEN, o ciclo normalmente prossegue com a refatoração: melhorar nomes, eliminar duplicidades e reorganizar o código sem alterar o comportamento. Em seguida, um novo teste inicia outro ciclo RED-GREEN. Essa continuidade mantém as regras verificáveis durante a evolução do projeto.

12. Implementação tecnológica

Camada	Tecnologia	Finalidade
Interface	Streamlit ≥ 1.53	Aplicação multipágina em Python.
Domínio	Python ≥ 3.10	Classes, abstração, herança, composição e regras.
Persistência	SQLite 3	Banco local transacional.
Valores	INTEGER em centavos	Evitar imprecisão de FLOAT.
Exportação	pandas + CSV	Cronograma interoperável de 12 meses.
Testes	pytest + Streamlit AppTest	Regras, banco, interface e exportação.

12.1 Estrutura principal

app.py - inicialização da aplicação
 pages/ - páginas da interface Streamlit
 models/ - entidades e objetos de domínio
 services/ - cálculo e exportação
 database/ - conexão, consultas e transações SQLite
 data/imobiliaria.db é criado automaticamente durante a execução e não precisa estar dentro do ZIP.

12.2 Diferenciais técnicos

Persistência em seis tabelas normalizadas, incluindo memória de cálculo e 12 parcelas.
 Transação atômica: uma falha impede registros parciais.
 Restrições CHECK, UNIQUE, chaves estrangeiras, índices e visão SQL.
 Valores monetários representados em centavos inteiros.
 Serviços independentes da interface, facilitando manutenção e testes.

12.3 Sequência entre as camadas

O usuário informa os dados na interface Streamlit.

A interface valida os campos e envia os dados ao CalculoService.

O serviço instancia a classe do imóvel selecionado e solicita os itens do cálculo. O Orcamento agrega cliente, imóvel, contrato, itens e

cronograma.

A camada de banco persiste o agregado dentro de uma transação.

O ExportService transforma o cronograma em DataFrame e gera os bytes do CSV.

13. Integridade SQLite

O banco registra clientes, imóveis, contratos, orçamentos, itens da memória de cálculo e parcelas mensais. A estrutura preserva o histórico exato do orçamento mesmo que regras futuras sejam alteradas.

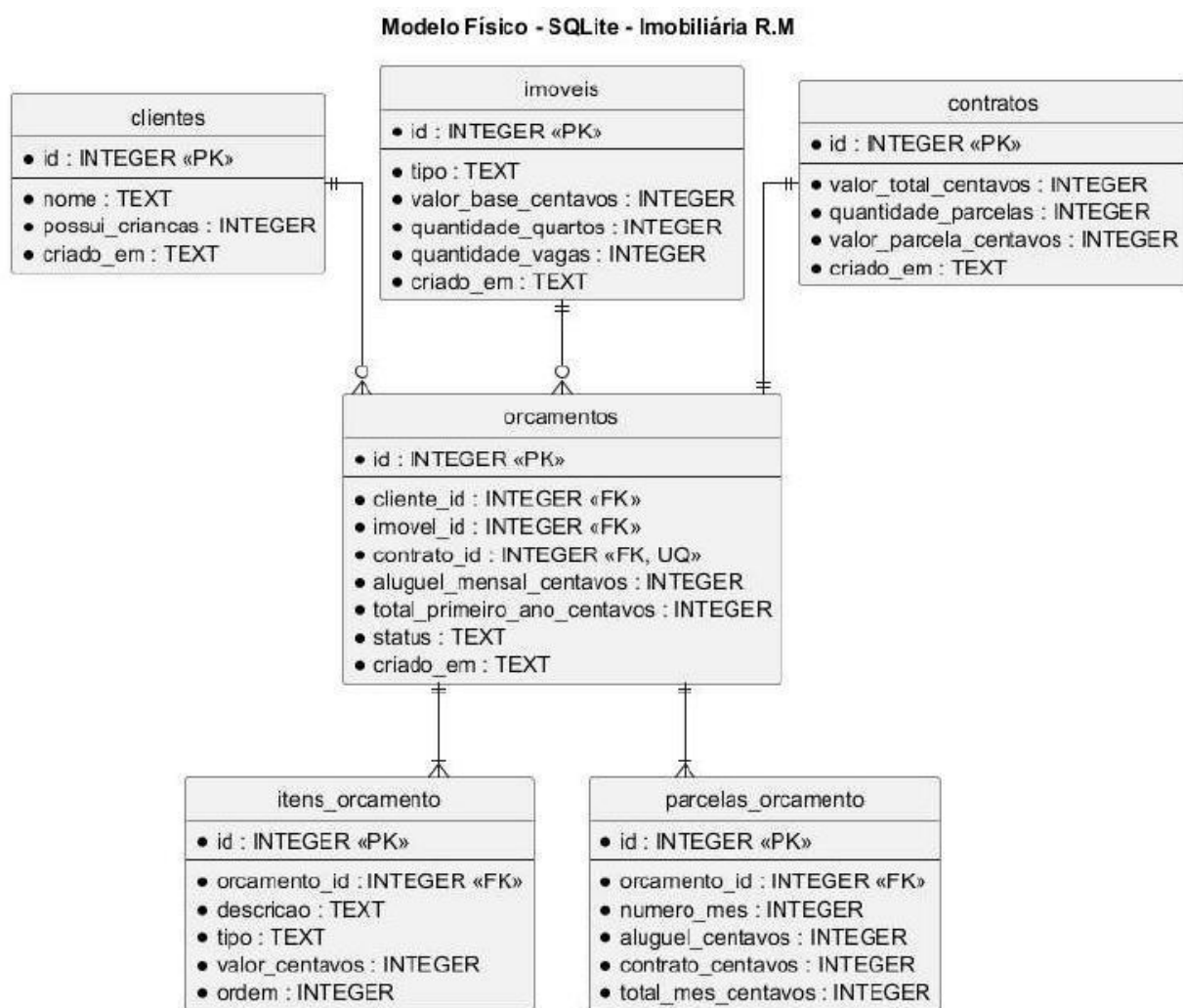


Figura 7 - Tabelas, chaves e cardinalidades físicas

Figura 3 - Modelo físico SQLite: tabelas, chaves e cardinalidades

13.1 Regras de integridade

Mecanismo	Aplicação
PK	Identificador autoincremental em todas as tabelas.
FK	Cliente, imóvel, contrato, itens e parcelas vinculados ao orçamento.

CHECK	Validação de tipos, booleanos, parcelas, meses, status e valores.
UNIQUE	Contrato 1:1, ordem dos itens e mês sem duplicidade.
CASCADE	Somente itens e parcelas são removidos com o orçamento.
RESTRICT	Entidades principais não são excluídas acidentalmente.
Índices	Cliente, data de criação e parcelas.
Visão	vw_orcamentos_resumo para painel e listagem.

14. Geração do CSV com 12 meses

O CSV representa os 12 meses do orçamento. O aluguel permanece em todos os meses, enquanto a parcela do contrato aparece somente durante a quantidade de meses escolhida, limitada a cinco. Depois disso, o valor do contrato no mês passa a zero.

14.1 Colunas sugeridas

numero_mes: posição do mês no cronograma, de 1 a 12.

aluguel_centavos: valor mensal do aluguel.

contrato_centavos: parcela do contrato aplicável naquele mês.

total_mes_centavos: soma do aluguel com a parcela contratual.

14.2 Exemplo: apartamento de R\$ 1.140,00 e contrato em cinco parcelas

Mês	Aluguel	Contrato	Total do mês
1	R\$ 1.140,00	R\$ 400,00	R\$ 1.540,00
2	R\$ 1.140,00	R\$ 400,00	R\$ 1.540,00
3	R\$ 1.140,00	R\$ 400,00	R\$ 1.540,00
4	R\$ 1.140,00	R\$ 400,00	R\$ 1.540,00
5	R\$ 1.140,00	R\$ 400,00	R\$ 1.540,00
6 a 12	R\$ 1.140,00	R\$ 0,00	R\$ 1.140,00

Nesse exemplo, o total do primeiro ano é R\$ 15.680,00: 12 × R\$ 1.140,00 de aluguel mais R\$ 2.000,00 do contrato.

14.3 Regras de exportação

Gerar exatamente 12 registros mensais, além do cabeçalho.

Utilizar codificação UTF-8 e separador compatível com o programa de planilha adotado.

Preservar os valores calculados no orçamento, sem recalculá-los durante a exportação.

Disponibilizar o arquivo somente após o orçamento ser validado e salvo com sucesso.

15. Testes e validação

Foram executados vinte testes automatizados envolvendo regras de negócio, validações, persistência, integridade, interface e exportação. As tabelas a seguir sintetizam nove verificações representativas, e os três cenários oficiais apresentaram os valores esperados.

Teste	Esperado	Obtido	Situação
Apartamento: 2 quartos, garagem e sem crianças	R\$ 1.140,00	R\$ 1.140,00	Aprovado
Casa: 2 quartos e garagem	R\$ 1.450,00	R\$ 1.450,00	Aprovado
Estúdio com quatro vagas	R\$ 1.570,00	R\$ 1.570,00	Aprovado
Contrato em cinco parcelas	R\$ 400,00	R\$ 400,00	Aprovado
Cronograma anual	12 meses	12 meses	Aprovado

Teste	Esperado	Obtido	Situação
Integridade referencial	Sem violações	Sem violações	Aprovado
PRAGMA integrity_check	ok	Ok	Aprovado
Interface Streamlit	Orçamento nº 1	Orçamento nº 1	Aprovado
Exportação CSV	Arquivo disponível	Arquivo disponível	Aprovado

15.1 Comandos de teste

```
python -m pytest -v
```

```
python -m pytest --maxfail=1 -q
```

15.2 Critérios de aprovação

Os resultados financeiros devem corresponder exatamente aos valores esperados em centavos.

Nenhum orçamento pode ser salvo parcialmente quando ocorrer uma falha.

As chaves estrangeiras e as restrições devem permanecer válidas.

O CSV deve conter todos os 12 meses e refletir os mesmos valores exibidos na interface.

16. Execução do projeto

No Windows, abra o PowerShell ou o terminal do VS Code na pasta do projeto e execute:

```
python -m venv .venv
```

```
.\venv\Scripts\Activate.ps1
```

```
python -m pip install --upgrade pip
```

```
python -m pip install -r requirements.txt
```

```
python -m pytest -v
```

```
python -m streamlit run app.py
```

Após a inicialização, o Streamlit informa no terminal o endereço local da aplicação. Não é necessário utilizar um caminho pessoal fixo; os comandos

funcionam a partir da pasta em que o projeto foi salvo.

16.1 Estrutura mínima da pasta compactada

app.py e demais arquivos .py do projeto.

Pastas pages, models, services, database e tests.

requirements.txt com as dependências necessárias.

README.md com objetivo, instalação, execução, testes e link do vídeo.

.gitignore para excluir .venv, __pycache__, bancos temporários e arquivos locais desnecessários.

Diagramas e relatório em PDF, conforme a orientação da atividade.

17. Entregáveis

Entregável	Conteúdo	Situação antes do envio
Parte teórica - 25%	Relatório em PDF com fluxograma, lógica e pensamento algorítmico.	Conferir PDF final
Parte prática - 50%	Pasta compactada com código funcional e link do GitHub.	Concluído — ZIP final e GitHub sincronizados.
Vídeo pitch - 25%	Vídeo de até quatro minutos publicado em plataforma aceita.	Inserir link

17.1 Links para preenchimento

<https://github.com/AntonioBenevaldo/rm-orcamento-aluguel>

Vídeo pitch: INSERIR AQUI O LINK DO VÍDEO ANTES DA ENTREGA.

17.2 Roteiro recomendado para o vídeo pitch

Apresentar o nome do projeto e o problema solucionado.

Explicar brevemente as tecnologias e a estrutura orientada a objetos. Demonstrar um orçamento de apartamento, casa ou estúdio.

Mostrar o cálculo do contrato e o cronograma de 12 meses. Exportar o CSV e apresentar rapidamente os testes aprovados. Encerrar informando onde o código está publicado.

18. Conclusão

O Sistema de Orçamento Imobiliário R.M atende às regras apresentadas no desafio e demonstra, de forma integrada, pensamento algorítmico e programação orientada a objetos. O problema foi decomposto em entradas, validações, decisões, cálculos, persistência e saídas. As classes específicas de imóveis encapsulam as regras, enquanto os serviços mantêm o cálculo e a exportação independentes da interface.

A utilização de centavos inteiros elimina inconsistências monetárias entre o código, o diagrama de classes e o banco SQLite. Os testes automatizados fornecem evidências de que os cenários oficiais, a integridade do banco, a interface e a geração do CSV funcionam conforme o esperado.

Com a inclusão do link do vídeo pitch e a conversão deste relatório para PDF, a entrega estará organizada de acordo com os três componentes obrigatórios definidos pela disciplina.

19. Referências

UNIFECAP. Algorithmic Thinking & Introduction to Object-Oriented Programming. Material da disciplina disponibilizado no Ambiente Virtual de Aprendizagem, 2026.

PYTHON SOFTWARE FOUNDATION. Python 3 Documentation. Disponível em: <https://docs.python.org/3/>. Acesso em: 19 jul. 2026.

STREAMLIT. Streamlit Documentation. Disponível em: <https://docs.streamlit.io/>. Acesso em: 19 jul. 2026.

THE PANDAS DEVELOPMENT TEAM. pandas Documentation. Disponível em: <https://pandas.pydata.org/docs/>. Acesso em: 19 jul. 2026.

SQLITE. SQLite Documentation. Disponível em: <https://www.sqlite.org/docs.html>. Acesso em: 19 jul. 2026.

PLANTUML. PlantUML Documentation. Disponível em: <https://plantuml.com/>. Acesso em: 19 jul. 2026.